

	UNIVERSIDAD NACIONAL DE SAN AGUSTIN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLD-001	Página: 1

GUÍA DE LABORATORIO

(formato docente)

INFORMACIÓN BÁSICA					
ASIGNATURA:	INTRODUCCION AL DESARROLLO DE NUEVAS PLATAFORMAS (E)				
TÍTULO DE LA PRÁCTICA:	Navegación entre pantallas: inicio de sesión y registro con Jetpack Compose				
NÚMERO DE PRÁCTICA:	3	AÑO LECTIVO:	2026B	NRO. SEMESTRE:	VIII
TIPO DE PRÁCTICA:	INDIVIDUAL				
	GRUPAL	X	MÁXIMO DE ESTUDIANTES	2	
FECHA INICIO:	15/09/2026	FECHA FIN:	19/09/2026	DURACIÓN:	100 minutos
RECURSOS A UTILIZAR: Android Studio (Quail o superior), Kotlin, Jetpack Compose, librería Navigation Compose, emulador Android o dispositivo físico (API 34 o superior)					
DOCENTE(s): Roxana Evelyn Limache Calatayud					

OBJETIVOS/TEMAS Y COMPETENCIAS	
OBJETIVOS: - Comprender el uso de Navigation Compose para gestionar múltiples pantallas dentro de una sola actividad. - Implementar pantallas independientes (Composables) para el inicio de sesión y el registro de un usuario. - Aplicar el paso de datos entre pantallas mediante NavController y argumentos de navegación.	
TEMAS: - Navigation Compose: NavController y NavHost - Definición de rutas (routes) y paso de argumentos entre pantallas - Formularios con múltiples campos de texto y validación básica	
COMPETENCIAS	C.a Aplica de forma flexible técnicas, métodos, principios, normas, estándares y herramientas de ingeniería necesarias para la construcción de software e implementación de sistemas de información.
	C.b Investiga nuevos modelos, metodologías, técnicas, herramientas y tecnologías por ser necesarias para mantener la vigencia en el desempeño profesional.
	C.c
	C.d

CONTENIDO DE LA GUÍA

	UNIVERSIDAD NACIONAL DE SAN AGUSTIN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLD-001	Página: 2

I. MARCO CONCEPTUAL

Revisar el siguiente recurso antes de la sesión:

- Video: "Navegación en Jetpack Compose con Navigation Compose"

<https://www.youtube.com/watch?v=nwdc1ct06TI>

- Android Developers, Navigation con Compose:

<https://developer.android.com/develop/ui/compose/navigation>

¿Por qué navegar entre pantallas en Compose?

A diferencia del enfoque clásico de Android (una Activity por pantalla, comunicadas mediante Intent), en Jetpack Compose lo habitual es construir la aplicación con una sola Activity y varias pantallas representadas como funciones @Composable. La librería Navigation Compose se encarga de mostrar la pantalla correcta y de pasar datos entre ellas, sin necesidad de crear una Activity nueva por cada pantalla.

NavController y NavHost

El NavController es el objeto que controla la navegación: se le indica a qué pantalla moverse y él se encarga de mostrarla. El NavHost es el contenedor donde se define el conjunto de pantallas disponibles (el grafo de navegación) y se asocia cada una con una ruta (route), que es simplemente un texto que la identifica, por ejemplo "login" o "registro".

Definición de rutas y navegación

Cada pantalla se registra dentro del NavHost con la función composable("ruta") { ... }. Para moverse de una pantalla a otra se llama a navController.navigate("ruta"); para regresar a la pantalla anterior se usa navController.popBackStack(). La Figura 1 ilustra esta relación aplicada a las pantallas de inicio de sesión y registro.

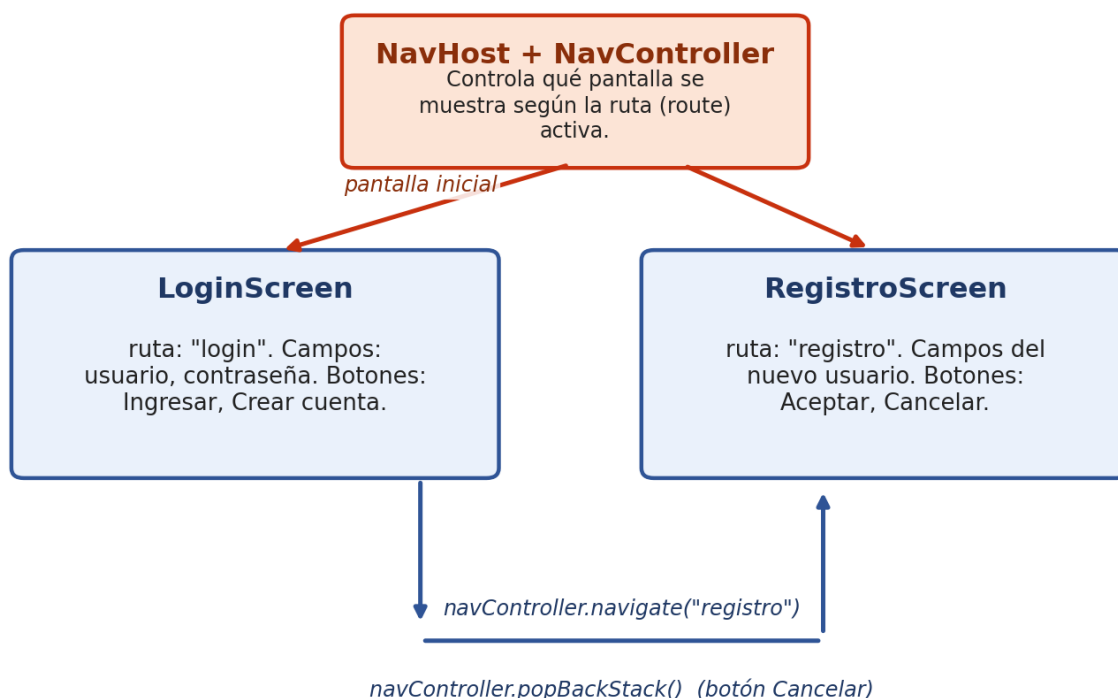


Figura 1. Grafo de navegación entre las pantallas de inicio de sesión y registro (elaboración propia).

	UNIVERSIDAD NACIONAL DE SAN AGUSTIN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLD-001	Página: 3

Paso de datos entre pantallas

Además de decidir qué pantalla mostrar, la navegación permite enviar información entre ellas: por ejemplo, los datos ingresados al registrar una cuenta pueden devolverse a la pantalla de inicio de sesión, o el resultado de validar las credenciales puede determinar el mensaje o la pantalla siguiente que se muestra al usuario. La Figura 2 resume el flujo de validación que se seguirá en el ejercicio resuelto.

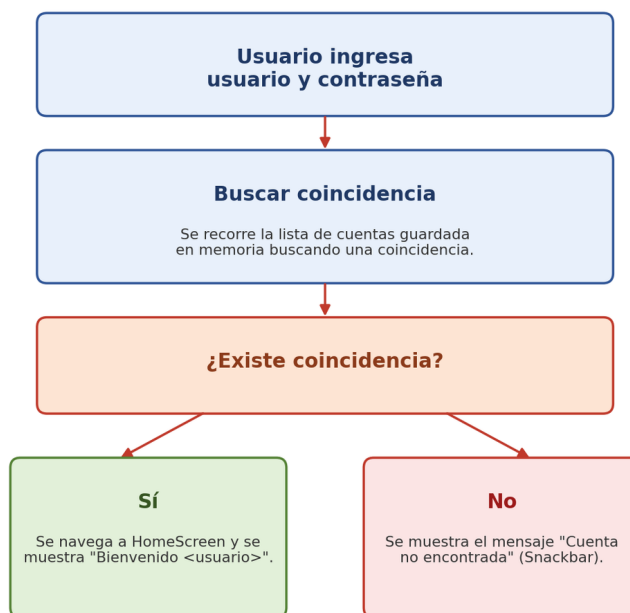


Figura 2. Flujo de validación al iniciar sesión (elaboración propia).

II. EJERCICIO/PROBLEMA RESUELTO POR EL DOCENTE

A continuación se detallan, paso a paso, las instrucciones para construir una navegación entre una pantalla de inicio de sesión y una de registro con Navigation Compose. También puede apoyarse en el siguiente video tutorial:

Video de referencia: "Navegación en Jetpack Compose con Navigation Compose"

<https://www.youtube.com/watch?v=nwdc1ct06TI>

Paso 1. Crear un nuevo proyecto con la plantilla "Empty Activity".

nombre sugerido para el proyecto: "NavCompose_LoginRegistro", Package name: com.example.navcompose_loginregistro, Language: Kotlin, Minimum SDK: el que venga por defecto en la plantilla (recuerden usar un emulador o dispositivo con API 34 o superior, según la sección de Recursos a utilizar).

Paso 2. Agregar la dependencia de Navigation Compose en build.gradle.kts (módulo app): implementation("androidx.navigation:navigation-compose:2.8.0") (o la versión estable más reciente), y sincronizar Gradle.

Código:

```
// build.gradle.kts (Module :app)
dependencies {
    implementation("androidx.navigation:navigation-compose:2.8.0")
    // ...el resto de dependencias que ya genera la plantilla
}
```

	UNIVERSIDAD NACIONAL DE SAN AGUSTIN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLD-001	Página: 4

Importaciones que se usarán en los siguientes pasos (agregar al inicio de MainActivity.kt):

```
import android.os.Bundle
import android.widget.Toast
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
import androidx.compose.foundation.layout.*
import androidx.compose.material3.*
import androidx.compose.runtime.*
import androidx.compose.ui.Modifier
import androidx.compose.ui.text.input.PasswordVisualTransformation
import androidx.compose.ui.unit.dp
import androidx.navigation.compose.NavHost
import androidx.navigation.compose.composable
import androidx.navigation.compose.rememberNavController
```

Paso 3. Crear dos funciones @Composable: LoginScreen(onLoginExitoso, onIrARegistro) y RegistroScreen(onRegistroExitoso, onCancelar). Cada una recibe funciones lambda para comunicar lo ocurrido, en lugar de manejar la navegación internamente.

Código(lista en memoria + esqueletos de las dos pantallas):

```
// Lista de cuentas registradas, compartida en memoria durante la ejecución
val cuentasRegistradas = mutableStateListOf(
    "admin" to "1234" // cuenta de prueba para el login
)

@Composable
fun LoginScreen(
    onLoginExitoso: (String) -> Unit,
    onIrARegistro: () -> Unit
) {
    // Contenido en el Paso 4
}

@Composable
fun RegistroScreen(
    onRegistroExitoso: () -> Unit,
    onCancelar: () -> Unit
) {
    // Contenido en el Paso 5
}
```

Paso 4. Dentro de LoginScreen, agregar dos TextField (usuario y contraseña, esta última con visualTransformation = PasswordVisualTransformation()) y dos botones: "Ingresar" y "Crear cuenta".

Código sugerido para LoginScreen:

```
@Composable
fun LoginScreen(
    onLoginExitoso: (String) -> Unit,
    onIrARegistro: () -> Unit
) {
    var usuario by remember { mutableStateOf("") }
    var password by remember { mutableStateOf("") }
    var mensajeError by remember { mutableStateOf("") }

    Column(
        modifier = Modifier
            .fillMaxSize()
            .padding(24.dp),
        verticalArrangement = Arrangement.Center
    ) {
        Text("Iniciar sesión", style = MaterialTheme.typography.headlineSmall)
        Spacer(modifier = Modifier.height(16.dp))
```

	UNIVERSIDAD NACIONAL DE SAN AGUSTIN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLD-001	Página: 5

```

        OutlinedTextField(
            value = usuario,
            onValueChange = { usuario = it },
            label = { Text("Usuario") },
            modifier = Modifier.fillMaxWidth()
        )
        Spacer(modifier = Modifier.height(8.dp))

        OutlinedTextField(
            value = password,
            onValueChange = { password = it },
            label = { Text("Contraseña") },
            visualTransformation = PasswordVisualTransformation(),
            modifier = Modifier.fillMaxWidth()
        )
        Spacer(modifier = Modifier.height(16.dp))

        if (mensajeError.isNotEmpty()) {
            Text(mensajeError, color = MaterialTheme.colorScheme.error)
            Spacer(modifier = Modifier.height(8.dp))
        }

        Button(
            onClick = { onLoginExitoso(usuario) }, // se completa en el Paso 8
            modifier = Modifier.fillMaxWidth()
        ) {
            Text("Ingresar")
        }
        Spacer(modifier = Modifier.height(8.dp))

        OutlinedButton(
            onClick = onIrARegistro,
            modifier = Modifier.fillMaxWidth()
        ) {
            Text("Crear cuenta")
        }
    }
}

```

Paso 5. Dentro de RegistroScreen, agregar los TextField necesarios para los datos del nuevo usuario, y dos botones: "Aceptar" y "Cancelar".

Código para RegistroScreen:

```

@Composable
fun RegistroScreen(
    onRegistroExitoso: () -> Unit,
    onCancelar: () -> Unit
) {
    var usuario by remember { mutableStateOf("") }
    var password by remember { mutableStateOf("") }

    Column(
        modifier = Modifier
            .fillMaxSize()
            .padding(24.dp),
        verticalArrangement = Arrangement.Center
    ) {
        Text("Crear cuenta", style = MaterialTheme.typography.headlineSmall)
        Spacer(modifier = Modifier.height(16.dp))

        OutlinedTextField(
            value = usuario,
            onValueChange = { usuario = it },
            label = { Text("Nuevo usuario") },

```

	UNIVERSIDAD NACIONAL DE SAN AGUSTIN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLD-001	Página: 6

```

        modifier = Modifier.fillMaxWidth()
    )
    Spacer(modifier = Modifier.height(8.dp))

    OutlinedTextField(
        value = password,
        onChange = { password = it },
        label = { Text("Nueva contraseña") },
        visualTransformation = PasswordVisualTransformation(),
        modifier = Modifier.fillMaxWidth()
    )
    Spacer(modifier = Modifier.height(16.dp))

    Button(
        onClick = {
            cuentasRegistradas.add(usuario to password)
            onRegistroExitoso()
        },
        modifier = Modifier.fillMaxWidth()
    ) {
        Text("Aceptar")
    }
    Spacer(modifier = Modifier.height(8.dp))

    OutlinedButton(
        onClick = onCancel,
        modifier = Modifier.fillMaxWidth()
    ) {
        Text("Cancelar")
    }
}

```

Paso 6. En MainActivity, dentro de setContent { }, crear el controlador con val navController = rememberNavController() y declarar NavHost(navController, startDestination = "login") registrando ambas pantallas: composable("login") { LoginScreen(...) } y composable("registro") { RegistroScreen(...) }, como se muestra en la Figura 1.

Código sugerido para MainActivity.kt:

```

class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContent {
            val navController = rememberNavController()

            NavHost(navController = navController, startDestination = "login") {
                composable("login") {
                    LoginScreen(
                        onLoginExitoso = { usuario ->
                            Toast.makeText(
                                this@MainActivity,
                                "Bienvenido $usuario",
                                Toast.LENGTH_SHORT
                            ).show()
                        },
                        onIrARegistro = {
                            navController.navigate("registro")
                        }
                    )
                }
                composable("registro") {
                    RegistroScreen(
                        onRegistroExitoso = {

```

	UNIVERSIDAD NACIONAL DE SAN AGUSTIN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLD-001	Página: 7

```

        navController.popBackStack()
    },
    onCancelar = {
        navController.popBackStack()
    }
)
}
}
}
}
}
}
}
}
}
}

```

Paso 7. Conectar la navegación: el botón "Crear cuenta" de LoginScreen debe llamar a `navController.navigate("registro");` el botón "Cancelar" de RegistroScreen debe llamar a `navController.popBackStack();`

Código (ya incluido en el bloque del Paso 6; se resalta aquí para mayor claridad):

```

onIrARegistro = { navController.navigate("registro") }
onCancelar = { navController.popBackStack() }

```

Paso 8. Implementar una validación básica en LoginScreen: al presionar "Ingresar", comparar los valores ingresados contra una lista de cuentas guardada en memoria (por ejemplo, una `List<Pair<String, String>>`). Si coincide, mostrar "Bienvenido <usuario>"; si no, mostrar un mensaje de error, siguiendo el flujo resumido en la Figura 2.

Código (reemplaza el bloque `Button(...)` "Ingresar" del Paso 4 por este):

```

Button(
    onClick = {
        val coincide = cuentasRegistradas.any {
            it.first == usuario && it.second == password
        }
        if (coincide) {
            mensajeError = ""
            onLoginExitoso(usuario)
        } else {
            mensajeError = "Usuario o contraseña incorrectos"
        }
    },
    modifier = Modifier.fillMaxWidth()
) {
    Text("Ingresar")
}

```

Paso 9. Ejecutar el proyecto y verificar el flujo completo: iniciar en LoginScreen, navegar a RegistroScreen con "Crear cuenta", regresar con "Cancelar", e iniciar sesión con credenciales válidas e inválidas.

Este paso no requiere código adicional; basta con ejecutar (Run) y probar el flujo completo.

Referencia de apoyo: <https://developer.android.com/develop/ui/compose/navigation>

III. EJERCICIOS/PROBLEMAS PROPUESTOS

Tomando como referencia el ejercicio resuelto por el docente, completa la aplicación de inicio de sesión y registro agregando persistencia de datos y una pantalla de bienvenida:

1. Agrega una tercera pantalla, HomeScreen(usuario), registrada en el NavHost con la ruta "home", que reciba el nombre del usuario como argumento de navegación (por ejemplo `navController.navigate("home/$usuario")`) y lo muestre en el mensaje "Bienvenido <usuario>".

	UNIVERSIDAD NACIONAL DE SAN AGUSTIN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLD-001	Página: 8

2. En RegistroScreen, al presionar "Aceptar", guarda los datos ingresados (usuario y contraseña) en un archivo de texto plano cuentas.txt en el almacenamiento interno de la aplicación (openFileOutput, modo MODE_APPEND), y regresa a LoginScreen mostrando un mensaje de confirmación.
3. En LoginScreen, al presionar "Ingresar", busca una coincidencia en el archivo cuentas.txt (en lugar de la lista en memoria usada en el ejercicio resuelto). Si existe, navega a HomeScreen; si no, muestra el mensaje "Cuenta no encontrada".
4. (Reto opcional) Agrega validación de campos vacíos en ambos formularios, mostrando un mensaje de error si el usuario intenta enviar el formulario incompleto.

Entregable: informe con las principales capturas de pantalla de las tres pantallas funcionando y una breve descripción de la implementación.

IV. CUESTIONARIO

1. ¿Qué ventajas tiene usar Navigation Compose (una sola Activity con varias pantallas) frente al enfoque tradicional de múltiples Activities comunicadas por Intent?
2. Además de los argumentos de navegación, ¿qué otro mecanismo podrías usar para compartir datos entre pantallas en una app Compose (por ejemplo, un ViewModel compartido)? Explica brevemente cómo funcionaría.

V. REFERENCIAS Y BIBLIOGRAFÍA RECOMENDADAS:

- [1] Android Developers, "Navigation con Compose". [En línea]. Disponible: <https://developer.android.com/develop/ui/compose/navigation>
- [2] Android Developers, "Aprende Android". [En línea]. Disponible: <https://developer.android.com/teach>
- [3] Android Developers, "Guías". [En línea]. Disponible: <https://developer.android.com/guide>
- [4] "Navegación en Jetpack Compose con Navigation Compose", video. [En línea]. Disponible: <https://www.youtube.com/watch?v=nwdc1ct06TI>

TÉCNICAS E INSTRUMENTOS DE EVALUACIÓN

TÉCNICAS: Ejercicios propuestos	INSTRUMENTOS: Rúbrica de evaluación
---	---

CRITERIOS DE EVALUACIÓN

- Cumplimiento de los objetivos y ejercicios propuestos en la guía.
- Correcta aplicación de los conceptos y herramientas del tema desarrollado.
- Funcionamiento correcto de lo implementado (compilación/ejecución sin errores).
- Informe (formato estudiante) completo, con evidencias y cuestionario resuelto.
- Entrega dentro del plazo establecido.